AI Crash Course for Tomorrow's Hackathon (TCS RT Problem Solving)

Zero-to-functional in one night. Goal: not to "learn AI" broadly, but to know enough patterns + have working code so that whatever problem drops tomorrow, you can move fast.

1. Vocabulary — just enough to not get lost

Term	What it actually means	When it matters to you
AI	Umbrella term: any system that mimics intelligent behavior	The whole event
ML (Machine Learning)	Algorithms that learn patterns from data instead of being hard-coded	Prediction/classification problems
DL (Deep Learning)	ML using neural networks (many layers)	Image/audio/complex pattern problems
LLM (Large Language Model)	A model trained on huge text data that predicts/generates text (GPT, Claude, Gemini, Llama)	Almost every hackathon problem can lean on this
Embedding	Turning text/image into a list of numbers (a vector) that captures meaning, so similar things end up numerically close	Search, recommendations, RAG
RAG (Retrieval-Augmented Generation)	Look up relevant info from your own data, then feed it to an LLM so it answers using that info instead of guessing	"Chatbot that knows our documents/data" — extremely common ask
Fine-tuning	Retraining a model on your own data	**Avoid this in a hackathon** — too slow, too much data/compute needed. Almost never the right call in <24-48 hrs.
Agent	An LLM that can call tools/functions/APIs in a loop to complete multi-step tasks	"Automate this workflow" type problems

Term	What it actually means	When it matters to you
Prompt engineering	Crafting the input text to get reliable, correct output from an LLM	Everything you build

The single most important rule for a hackathon: never train a model from scratch, and almost never fine-tune. Use **pretrained models via API** or **small pretrained models from Hugging Face.** Your job in 24-48 hours is integration and product thinking, not research.

2. The Problem → Solution Map

Whatever the problem statement says tomorrow, it'll roughly match one of these. Memorize the "go-to stack" column.

If the problem is about...	Pattern	Go-to stack
Answering questions from documents/data/policies	**RAG**	LLM API + embeddings + FAISS/Chroma (vector store)
A chatbot / assistant / customer support	**LLM + system prompt**, maybe RAG	LLM API + Streamlit chat UI
Predicting a number or category from structured data (sales, churn, risk score)	**Classical ML**	`pandas` + `scikit-learn` (Logistic Regression / Random Forest / XGBoost)
Images (defect detection, classification, OCR)	**Pretrained vision model**	Hugging Face `transformers`/`ultralytics` (YOLO) — don't train from scratch
Summarizing text/reports/tickets	**LLM with a summarization prompt**	LLM API, straightforward
Automating a multi-step process (extract → decide → act)	**Agent / tool-calling**	LLM API with function calling
Recommending items (products, content)	**Similarity/embeddings** or simple collaborative filtering	Embeddings + cosine similarity, or `scikit-learn`
Real-time data / dashboards	**Not really "AI"** — this is a trap teams fall into. Build the dashboard, add ONE smart feature (e.g. anomaly flag or LLM-generated insight) on top	`pandas`, `plotly`/`streamlit`, plus one LLM call for "explain this trend"

TCS "real-time problem solving" hackathons are usually enterprise-flavored (ops efficiency, customer service, document processing, fraud/anomaly detection, supply chain, HR/IT helpdesk automation). RAG + chatbot and classification/anomaly-detection problems are very common in this space — if you only prep two things deeply, prep those.

3. Getting an LLM API working (do this tonight, not tomorrow)

Pick ONE now and get a key so you're not fighting signup forms during judging:

OpenAI (`platform.openai.com`) — most tutorials/examples assume this
Google Gemini (`aistudio.google.com`) — has a generous free tier

Groq (`console.groq.com`) — free, extremely fast, good for live demos
If your team already has Anthropic/Claude API access, that works identically

Minimal call (swap client for whichever provider you pick — APIs are ~95% the same shape):

```python
from openai import OpenAI
client = OpenAI(api_key="YOUR_KEY")

response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[
    {"role": "system", "content": "You are a helpful assistant for X."},
    {"role": "user", "content": "Summarize this: ..."}
  ]
)
print(response.choices[0].message.content)
```

Prompting rules that actually matter

Give a role/system prompt. "You are a support agent for a telecom company. Only answer using the provided context." beats a bare question every time.
Show, don't just tell. Give 1-2 example input→output pairs in the prompt if format matters (few-shot).
Force structured output when you need to parse the result in code:
"Respond ONLY with valid JSON in this exact format: {"category": "...", "confidence": 0-1}"
Break big asks into steps. Instead of "solve the whole problem," chain smaller prompts (extract → classify → generate response).
Always show your context. For RAG: "Answer using ONLY the following context. If not found, say you don't know." — this stops hallucination, which judges will test.

4. RAG in ~20 lines (the pattern you'll most likely need)

```python
```

```python
import faiss
from sentence_transformers import SentenceTransformer

# 1. Load your data (docs, FAQs, tickets — whatever the problem gives you)
documents = ["doc text 1", "doc text 2", "..."]

# 2. Embed it
embedder = SentenceTransformer('all-MiniLM-L6-v2')  # free, local, no API needed
doc_vectors = embedder.encode(documents)

# 3. Build a searchable index
index = faiss.IndexFlatL2(doc_vectors.shape[1])
index.add(doc_vectors)

# 4. At query time: find relevant chunks
def retrieve(query, k=3):
    q_vec = embedder.encode([query])
    _, indices = index.search(q_vec, k)
    return [documents[i] for i in indices[0]]

# 5. Feed retrieved chunks + query to the LLM
context = "\n".join(retrieve("user's question"))
prompt = f"Context:\n{context}\n\nQuestion: user's question\n\nAnswer using only the context above."
```

`sentence-transformers` + `faiss-cpu` run fully locally and free — no API needed for the retrieval step, only for the final generation call. Install:

```bash
pip install sentence-transformers faiss-cpu openai streamlit pandas scikit-learn
```

5. Classical ML in ~10 lines (for prediction/classification problems)

```python
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("data.csv")
X = df.drop("target_column", axis=1)
y = df["target_column"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier()
model.fit(X_train, y_train)
print("Accuracy:", accuracy_score(y_test, model.predict(X_test)))
```

Random Forest is a great hackathon default: minimal tuning, handles messy data, hard to badly mess up.

6. Quick UI (so you have a demo, not just a script)

```
bash
pip install streamlit
```

```
python
import streamlit as st

st.title("Your Solution Name")
user_input = st.text_input("Ask something:")
if user_input:
    answer = "call your function here"
    st.write(answer)
```

Run with `streamlit run app.py`. This alone gets you a clickable demo in under 5 minutes — judges respond to seeing something working, not just code.

7. Tonight's build (do this before sleeping)

Pick RAG or classification (whichever feels closer to what you expect tomorrow — RAG/chatbot is the safer

bet for TCS-style problems) and get ONE of the code blocks above **actually running** on your machine with fake/sample data tonight. Debug the environment issues now (installs, API keys, Python version mismatches) — not during the hackathon clock.

8. Tomorrow's time-box (once the problem is revealed)

Time	Do
First 20-30 min	Read the problem twice. Map it to a pattern from Section 2. Resist the urge to start coding immediately.
Next 30 min	Sketch the simplest version that "works end to end" — even badly. A working ugly demo beats a beautiful half-built one.
Bulk of remaining time	Build the core pipeline, then the UI last.
Last hour	Stop building new features. Polish the demo flow and prep your pitch.

9. Pitch tips (judges often weigh this heavily)

Open with the **problem/pain point**, not the tech.
Show a **live demo**, not slides of code.
Be upfront about one limitation and how you'd fix it with more time — this reads as maturity, not weakness.
Mention *why* you chose your approach (e.g. "we used RAG instead of fine-tuning because it's faster and avoids hallucination risk") — judges like hearing you made a deliberate trade-off.

One-line mantra for tomorrow

Don't train models. Retrieve, prompt, and glue APIs together. Ship an ugly working thing early, then polish.